

OpenLoopEvolve: A Verifiable Self-Evolution Framework for Loop Policies in Long-Horizon Complex Tasks

Siqi Wang*, Xinlin Li*, Zhenglin Li, and Li Li

Abstract—Long-horizon complex tasks require agents to repeatedly observe states, formulate plans, invoke tools, verify results, and recover from failures in continuously changing environments. However, such control experience often remains confined to a single context or a fixed prompt, and is difficult to accumulate and reuse across historical traces. This paper presents OpenLoopEvolve (OLE), a self-evolution framework centered on the Loop Policy. OLE represents an agent’s observation, planning, memory, action, verification, recovery, stopping, and budget control as portable policy assets with versions and lineages, and provides online and offline evolution modes that can be selected according to practical needs: the online mode triggers candidate generation based on feedback from continuous operation, whereas the offline mode searches for candidate policies from archived traces and failure evidence. Both modes share an evolution mechanism consisting of autonomous proposals by a large language model, Champion–Challenger paired evaluation, and robust release. Policies released online are activated at a subsequent task boundary, monitored using subsequent feedback, and rolled back to their parent versions when degradation conditions are met. On the simulated business benchmark YC-Bench, both modes improve aggregate task performance, task success rate, and risk metrics relative to a fixed initial Loop Policy. The results indicate that treating the Loop Policy as a governable asset can support the accumulation, comparison, release, and reuse of control experience and improve agent performance on long-horizon complex tasks.

Index Terms—large language model agents, long-horizon tasks, Loop Policy, self-evolution

I. INTRODUCTION

Large language model agents are evolving from tools that generate one-shot answers into task executors capable of sustained action in external environments. Across a sequence of decisions in a long-horizon complex task, an agent must acquire observations, revise plans, invoke tools, verify results, and, after a local failure, decide whether to retry, recover, or stop. Interleaving reasoning and action allows agents to adjust their behavior based on environmental feedback [1], while interactive evaluation has begun to expand from short processes to cross-page operations and stateful business workflows [2]–[5], as well as the simulated year-long business management studied by YC-Bench [6]. In these tasks, stopping prematurely may leave unverified artifacts, whereas repeated planning and

ineffective calls consume resources needed later; moreover, a state change caused by one action may invalidate subsequent steps in the original plan. Observation granularity, recovery methods, and budget allocation therefore form interdependent control relationships, and the final outcome depends on the entire execution loop rather than only on single-step generation quality.

Existing studies use external memory, linguistic reflection, and experience extraction to reuse past events or failures in subsequent decisions [7]–[9]; executable skill libraries further support the retrieval and composition of procedural behaviors [10]. Meanwhile, declarative pipelines, code-based workflows, and modular agents bring structures external to the model into automated optimization [11]–[14], and trajectory feedback has also been used to continually revise multi-step reasoning processes [15]. These directions improve experiential content and execution structure, respectively, but they have not represented the overall control relationships among observation, planning, memory use, verification, recovery, stopping, and budget allocation as a unified object. Memory can provide content from the past, and skills can reproduce local operations, but neither inherently specifies when to invoke experience, how to check execution results, or when to terminate. Although historical traces can be stored or retrieved, their successes and failures remain difficult to convert into comparable and transferable modifications to the complete loop.

The recently emerging perspective of Loop Engineering treats triggers, goals, verification, stopping rules, and memory organization as design elements of loops external to the model [16], providing a new level of analysis for the above problem. A loop object that can accumulate across runs also requires stable control semantics, host interfaces, and version relationships so that the same modification has comparable meaning across different runs. Loop optimization must also separate candidate generation from formal release: immediately rewriting the loop without candidate validation may introduce incidental noise or tail degradation into subsequent tasks, whereas always retaining a fixed loop prevents the system from learning from recurring failures. Research on evidence gating has highlighted the distinction between generated claims and verification states [17]; Champion–Challenger mechanisms also constrain version replacement through candidate comparisons in predictive-model management and online AutoML [18], [19]. However, these mechanisms have not been applied directly to complete agent loops, nor do they provide a shared policy asset and release semantics for offline historical trajectories and online operational feedback.

Against this background, this paper focuses on two related questions: first, how to separate the complete execution loop

*Siqi Wang and Xinlin Li contributed equally to this work and share first authorship.

Siqi Wang and Li Li are with the Department of Automation, Tsinghua University (e-mail: wang-sq24@mails.tsinghua.edu.cn; li-li@mail.tsinghua.edu.cn).

Xinlin Li is with the Department of Industrial Engineering, Tsinghua University (e-mail: lix122@mails.tsinghua.edu.cn).

Zhenglin Li is with Shenzhen International Graduate School, Tsinghua University (e-mail: li-zl23@tsinghua.org.cn).

Corresponding author: Li Li.

OpenLoopEvolve Framework

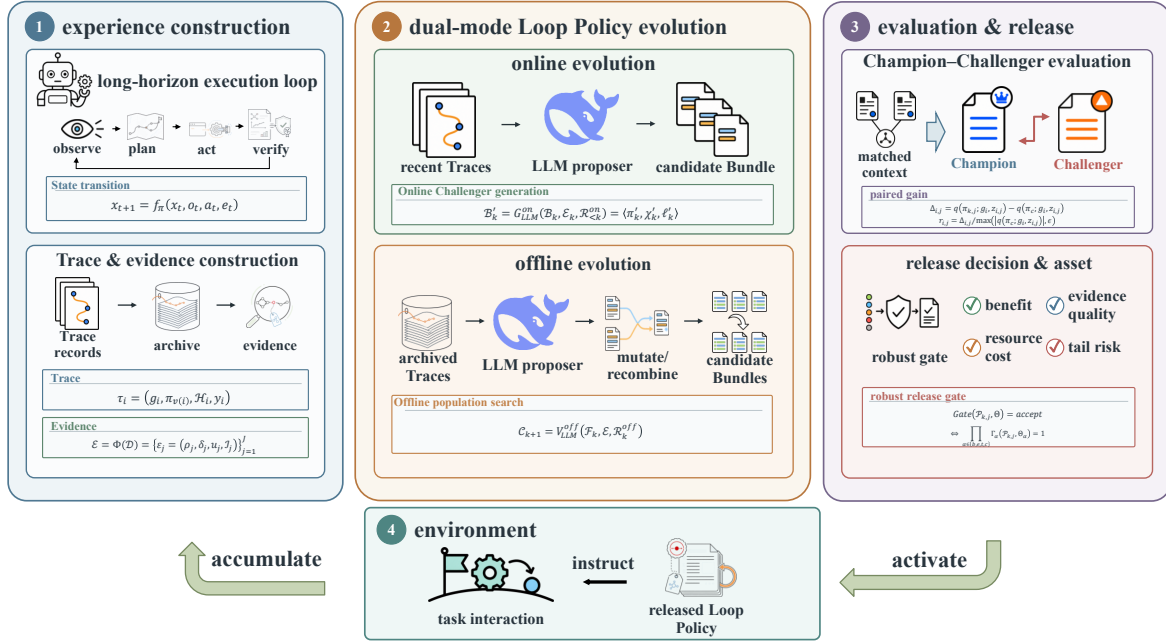


Fig. 1. Overall OpenLoopEvolve framework. A released Loop Policy is activated at a subsequent task boundary, enters the host environment, and controls interactions in long-horizon tasks, producing attributable traces. The traces are converted into evolution evidence under a shared evidence contract and enter online or offline Loop Policy evolution. Candidates are evaluated against the Champion under identical conditions and, after passing the robust gate, are released as versioned policy assets and activated at a task boundary.

from the host implementation and a single context to form an external policy asset that can be versioned, ported, compared, and reused; and second, how to autonomously improve this asset using historical traces and current feedback while allowing only policies that have passed candidate validation, been released, and been activated at a task boundary to affect future runs. Online updates need to avoid disrupting the state of a task currently under execution, whereas offline search needs to form comparable candidate evaluations from multiple heterogeneous traces. The two modes thus differ in when evidence becomes available but require consistent asset and release semantics. The first question determines the object that serves as the unit of loop evolution, and the second determines how evidence drives policy changes and when those changes can be released and activated. The objective of this paper is not to allow an agent to rewrite itself arbitrarily within a single run, but to place policy search outside the execution path so that candidates, evaluation evidence, and formal versions remain clearly separated.

To address these questions, this paper presents OpenLoopEvolve (OLE), a verifiable self-evolution framework for Loop Policies in long-horizon complex tasks. OLE organizes observation, planning, memory, action, verification, recovery, stopping, and budget control into an externalized Loop Policy, and encapsulates the policy, its applicability conditions, and its version provenance in a Bundle within a policy lineage. The Loop Policy constrains a run through general control semantics without binding it to the specific model and tool implementation of the host, enabling loop experience to be

read, compared, and reused independently of the original trace. A run’s trace is then distilled into traceable evidence that points to policy modifications, after which a large language model autonomously proposes candidates.

OLE provides online and offline evolution modes according to the evidence source. The online mode generates a single Challenger based on recent feedback; following paired evaluation and a robust gate, the Challenger is released as a successor asset, activated at a subsequent task boundary, and then subject to degradation monitoring and rollback. The offline mode constructs a candidate population from archived traces and performs multi-generation search through batch evaluation, elite retention, and autonomous mutation and recombination. The two modes can be adopted separately according to run continuity, budget, and candidate-validation timing, while sharing the boundary of “evidence–candidate–candidate validation–release–activation”: candidate generation belongs to the search process, only policies satisfying the candidate-validation requirements can be released, and only a released policy activated at a task boundary can enter the execution path. This forms a closed loop from historical interactions to subsequent policies, as illustrated in Fig. 1, allowing control experience to evolve continually while preventing provisional candidates from directly altering the execution path.

The main contributions of this paper are summarized as follows:

- 1) We propose the Loop Policy and Bundle protocol, which represents complete loop control as an external policy asset with explicit interfaces, versions, and lineages,

enabling historical control experience to be preserved, compared, exchanged, and reused across runs.

- 2) We propose online and offline self-evolution methods based on trace evidence. A large language model autonomously proposes candidates, while paired evaluation, robust release, task-boundary activation, and roll-back govern their entry into the execution path. To our knowledge, this work is the first to apply the Champion–Challenger release mechanism to both offline and online self-evolution of externalized Loop Policies.
- 3) We evaluate OLE on YC-Bench. Results obtained with fixed models, benchmark configurations, and official seeds show that both online and offline modes outperform the fixed initial Loop Policy in long-term returns, task success rate, annual survival, and risk metrics.

The remainder of this paper is organized as follows. Section II reviews research on long-horizon interactive agents, reuse of memory and trajectory experience, agent workflow optimization, and self-evolution. Section III defines long-horizon complex tasks, the loop interaction process, the trace, evolution evidence, Loop Policy, and Bundle. Section IV introduces the shared evolution framework of OpenLoopEvolve and its online and offline implementations. Section V presents the experimental results and analysis. Section VI concludes the paper. The code for this paper can be found at <https://github.com/yoyoshikc/OpenLoopEvolve>.

II. RELATED WORK

A. Long-Horizon Interactive Agents and Loop Execution

Multi-round interaction with an environment is a fundamental capability of large language model agents in complex tasks. Unlike one-shot answer generation, an agent must receive environment states, select actions, and use new feedback to advance its objective over a sequence of decisions; task performance therefore depends on consistent reasoning and execution across steps. Existing comprehensive benchmarks have examined these capabilities in diverse interactive environments and have further recorded agents’ intermediate progress before task completion [2], [3].

One line of research on this process constructs an execution loop in which reasoning and action are interleaved. Yao et al. [1] organize language reasoning, external actions, and environment observations within the same trajectory, enabling the model to adjust subsequent judgments based on execution outcomes. Another line of research embeds such loops in workflows involving real state changes: Zhou et al. [4] evaluate cross-page operations and functional correctness in web environments, while Yao et al. [5] further evaluate task completion in interactions involving tools, users, and domain states. Correspondingly, long-horizon benchmarks have also begun to emphasize the cumulative consequences of control decisions: He et al. [6] test long-term planning and consistent execution through simulated business operations sustained over a year, rather than evaluating only a single-round response.

These studies establish the closed-loop interaction paradigm for long-horizon tasks and demonstrate the roles of planning, tool use, and state feedback in task completion. Their primary

objective, however, remains to improve or evaluate agent behavior within a given execution structure; how observations are organized, when planning or recovery occurs, how outcomes are verified, and when execution stops are typically determined by the particular method or host framework. This leaves room to investigate how these control rules can be separated from the implementation details of an individual run and treated as a unified object for comparison and optimization. Enabling loop control rules to improve continually across runs also requires addressing how historical interactions can be accumulated and brought back into subsequent decisions.

B. Memory, Reflection, and Reuse of Trajectory Experience

Memory and reflection provide the main means for agents to accumulate experience across rounds. Their shared premise is to preserve historical interactions beyond the limited context window in an external medium and retrieve, summarize, or reorganize them during subsequent decisions, thereby maintaining consistency in long-term behavior. Park et al. [7] connect the recording of experiences with reflection and planning, enabling agents to derive higher-level insights from past events and use them to guide subsequent behavior; this demonstrates that historical trajectories can contribute to continual decision-making beyond serving as raw conversation records.

Existing methods differ in how they preserve and use experience. Shinn et al. [8] convert task feedback into linguistic reflections and store them in episodic memory, allowing subsequent trials to draw on failures without updating model parameters; Zhao et al. [9] extract natural-language experiences from a set of training tasks and jointly retrieve abstract insights and similar successful trajectories at inference time. In contrast to textual experience, Wang et al. [10] store successful programs generated from environment feedback in a retrievable skill library, allowing procedural behaviors to be composed and reused in new Minecraft worlds. These methods show, respectively, that historical trajectories can be distilled into reflective text, experiential knowledge, or executable skills.

These mechanisms substantially expand agents’ ability to reuse past experience, but their update targets are primarily memory content, linguistic experience, or task skills, and they generally do not directly represent the overall control relationships among observation, planning, verification, recovery, stopping, and budget allocation. This paper does not seek to add another form of memory content; instead, it distills evidence from traces that supports policy comparison and uses this evidence to modify how the complete loop is controlled. This distinction leads further to the question of agent workflows and their self-evolution.

C. Agent Workflow Optimization and Self-Evolution

As agent systems have expanded from single prompts into composite programs containing model calls, tools, and control flow, the object of study has gradually shifted from prompt optimization to the optimization of workflows and agent structures. Khattab et al. [11] represent language model pipelines as composable declarative modules and compile their

parameters against specified metrics; Hu et al. [12] instead represent agent systems as code, enabling a meta-agent to generate and search for new agent programs, with a code space that can in principle cover prompts, patterns of tool use, and workflow compositions. These studies show that execution structures external to the model can themselves become objects of automatic optimization.

In terms of the evidence used for optimization, existing work can be summarized along two paths: offline structural search and continual improvement driven by runtime feedback. Offline search uses existing tasks, execution evaluations, and accumulated search experience to produce candidate structures; Zhang et al. [13] model code-represented Agent Workflows as a search space and iteratively modify workflows using execution feedback and tree-structured experience; Shang et al. [14] abstract planning, reasoning, tool use, and memory into modules with unified interfaces and search for agent designs through evolution and recombination. By comparison, trajectory-driven self-evolution research places greater emphasis on using newly generated interaction feedback to adjust subsequent solution processes; Guo et al. [15] extract feedback from multi-step reasoning trajectories and iteratively optimize the reasoning process. The former focuses on searching for structures in controlled evaluation environments, whereas the latter focuses on allowing runtime experience to continually influence subsequent behavior; its focus nevertheless remains iterative optimization of the reasoning process and does not yet address policy release governance for deployment. Together, these two paths move agents from fixed designs toward experience-driven continual optimization.

The recent emergence of Loop Engineering further concentrates attention on loop structures external to the model. Macedo [16] organizes trigger conditions, objectives, verification, stopping rules, and memory into reusable loop specifications, while Huang et al. [17] emphasize that only evidence satisfying a verification gate can drive state transitions in the agent lifecycle. These studies extend workflow optimization to the systematic design of complete execution loops and begin to address verification and state governance within them. In related work, the Champion–Challenger paradigm has been used for the continual comparison and replacement of production predictive models [18] and has been further developed into a hyperparameter-selection algorithm that concurrently maintains one Champion and multiple online Challengers [19]. Existing methods, however, primarily compare predictive models or learning configurations. Loop Engineering itself has not yet established unified object boundaries and interface standards; most existing optimization results remain specific programs, task structures, or configurations internal to a framework, while offline search and online runtime feedback still lack a shared policy asset and release semantics. To address this gap, this paper represents loop control logic as a versioned and portable Loop Policy, uses a unified protocol to accommodate both offline and online candidates, and governs their evolution through Champion–Challenger comparison, evidence-constrained release, and rollback. To the best of our knowledge, this paper is the first to apply a Champion–Challenger release mechanism to the offline and online self-evolution of an

externalized Loop Policy, enabling loop-control experience from long-horizon tasks to be verified, released, and reused as an independent asset.

III. PROBLEM FORMULATION

A. Long-Horizon Tasks and the Loop-Control Problem

Sustained execution of multi-round tasks in interactive environments is an important test of an agent’s reasoning, decision-making, and execution capabilities, and has become a major focus of general-agent research and evaluation [2], [3]. Such tasks require an agent to engage in closed-loop interaction with an external environment over successive decision rounds toward a given objective: the agent selects an action based on its current observation, the action changes the environment state and produces new feedback, and subsequent decisions therefore depend on prior interactions. Existing research characterizes the long-horizon nature of these tasks primarily at two levels: solution mechanisms and evaluation protocols. At the solution-mechanism level, Yao et al. [1] organize task execution as closed-loop interaction, enabling an agent to update its plan in response to feedback and handle exceptions. At the evaluation-protocol level, WebArena and Tau-bench [4], [5] construct tasks as realistic interactive workflows with multistep dependencies and determine task completion from functional correctness or the post-execution environment state. The former reflects closed-loop decision-making and state dependence during execution, whereas the latter ensures that task outcomes can be verified through the external environment.

Following the closed-loop interaction perspective, we view the solution of a long-horizon complex task as an execution loop in which an agent repeatedly cycles through reasoning, action, and observation of the environment, and we focus on the mechanisms that control observation, planning, action, verification, recovery, and stopping within this loop. We define a long-horizon complex task as one whose solution process cannot be reliably reduced to a single model generation without environment interaction, but instead requires an agent to execute multiple consecutive decision steps with state dependencies under finite resource constraints, adjusting subsequent behavior according to tool returns and environment feedback until it produces a verifiable task outcome. Formally, we describe a long-horizon complex task using the following executable contract:

$$g = \langle d, \mathcal{U}_g, \mathcal{M}_g, \mathcal{S}_g, \mathcal{V}_g, \beta_g \rangle, \quad g \in \mathcal{G}. \quad (1)$$

Here, $\mathcal{G}$ denotes the space of long-horizon complex tasks. d describes the task objective, expected artifact, and initial conditions, providing an anchor for planning and replanning within the loop. $\mathcal{U}_g$ specifies the set of executors permitted by the task, thereby restricting the tool calls and environment actions available to the loop. $\mathcal{M}_g$ provides the evaluation metrics that determine how execution progress, outcome quality, and resource cost are measured. $\mathcal{S}_g$ restricts the evidence sources that may support a judgment, such as environment observations, run logs, and task artifacts, preventing task completion from being established solely by the model’s own statements. $\mathcal{V}_g$

specifies when and how verification is performed and feeds its results back into replanning, recovery, and stopping. β_g is the resource-budget vector comprising upper bounds on quantities such as interaction rounds, tool calls, time, and tokens, and thus bounds retries and exploration. These six fields specify the loop's objective, capability boundary, measurement criteria, grounds for judgment, verification mechanism, and resource boundary, respectively.

Based on this formal definition, we further characterize an agent's loop interaction process within a single run to make explicit the dynamic relationships among observations, internal states, actions, and environment feedback. Before giving its structured components, we use Π to denote the Loop Policy space abstractly and let $\pi \in \Pi$ denote the Loop Policy that constrains the current execution loop; its eight-component structure is defined in Section III-C. The loop-control process can be abstracted as

$$x_{t+1} = f_\pi(x_t, o_t, a_t, e_t), \quad a_{t+1} \sim h_\pi(x_{t+1}), \quad (2)$$

where t denotes the current interaction step; $x_t \in \mathcal{X}$ denotes the internal control state at step t ; $o_t \in \mathcal{O}$ denotes the observation that the agent receives from a tool or the environment at step t ; $a_t \in \mathcal{A}$ denotes the action executed at step t ; $\mathcal{E}_{\text{env}}$ denotes the environment-event space, and $e_t \in \mathcal{E}_{\text{env}}$ denotes the environment event caused by action a_t ; π denotes the Loop Policy that constrains loop behavior; f_π denotes the update function that, under Loop Policy π , computes the successor state from x_t , o_t , a_t , and e_t ; x_{t+1} denotes the next control state produced by f_π ; h_π denotes the decision function that selects an action from x_{t+1} ; and a_{t+1} denotes the next action produced by h_π . An action may be a tool call, information acquisition, artifact modification, outcome verification, or stopping decision, and its available range is jointly constrained by $\mathcal{U}_g$ in the task contract and the host's capabilities. The state x_t is an abstract representation of the control information required to run the loop and need not be a fully observable Markov state. Factors including observation, planning, memory use, reflection, risk handling, evaluation, and budget allocation affect subsequent state updates and action selection through f_π and h_π .

Having characterized the loop interaction process, we must also specify the basis for evaluating a Loop Policy and its optimization objective. Let $q(\pi; g, z) \in \mathbb{R}$ denote the value score attained by Loop Policy π on task g under controlled run condition $z \in \mathcal{Z}$, where $\mathcal{Z}$ denotes the run-condition space and q is determined by the evaluation metrics $\mathcal{M}_g$ and verification protocol $\mathcal{V}_g$ in the task contract. Under these conventions, Loop Policy optimization seeks, for the same task and controlled run condition, a Loop Policy with a higher value score that satisfies the verification requirements and resource boundary. This objective can be formalized as

$$\begin{aligned} \pi^* \in \arg \max_{\pi \in \Pi} \quad & q(\pi; g, z) \\ \text{s.t.} \quad & \mathbf{c}_{\mathcal{V}_g}(\pi; g, z) \preceq \kappa_g, \\ & \mathbf{b}(\pi; g, z) \preceq \beta_g, \end{aligned} \quad (3)$$

where Π is the Loop Policy space formally defined in Section III-C, π^* denotes an optimal Loop Policy for this

constrained optimization problem, $\mathbf{c}_{\mathcal{V}_g}(\pi; g, z)$ denotes the verification-risk vector determined by verification protocol $\mathcal{V}_g$, κ_g denotes the corresponding admissible upper bounds, $\mathbf{b}(\pi; g, z)$ denotes the resource-consumption vector, and $\preceq$ denotes elementwise vector comparison. This optimization problem jointly characterizes outcome quality, verification reliability, and resource boundaries. The observations, actions, and environment feedback produced during Loop Policy execution further form interaction traces, providing empirical grounds for analyzing existing Loop Policy behavior and supporting subsequent evolution. The next subsection accordingly defines a trace and its evidence form.

B. Traces and the Evidence Contract

The continual evolution of a Loop Policy depends on the effective use of historical runs, requiring the execution process to be formalized as a trace that links the task, Loop Policy, and outcome. Let the trace of the i th run be

$$\tau_i = (g_i, \pi_{v(i)}, \mathcal{H}_i, y_i), \quad \mathcal{H}_i = \{(o_t, a_t, e_t)\}_{t=1}^{T_i}, \quad (4)$$

where g_i denotes the task processed in the i th run, $v(i)$ denotes the stable version index of the registered Bundle used in that run, and $\pi_{v(i)}$ denotes the Loop Policy in that Bundle. The stable index $v(i)$ can be used to retrieve the complete asset $\mathcal{B}_{v(i)}$, including its applicability conditions and version provenance. $\mathcal{H}_i$ denotes the interaction history composed of observations, actions, and environment events, T_i denotes the trace length, and y_i denotes the run outcome. This representation explicitly preserves the mapping between policy-asset identity and the corresponding interaction outcome, ensuring that historical performance can be accurately attributed to a specific registered Bundle version.

After obtaining attributable traces, we must further distill from them evolution evidence that can guide adjustments to the Loop Policy. We define such evidence as a testable basis for modifying a specific Loop Policy component, inferred from a set of traces that are attributable, whose outcomes are verifiable, and whose evidence sources conform to the task contract. Each evidence item must specify the recommended direction of adjustment, the associated performance change, the policy component concerned, and the original traces supporting the judgment. Formally, the evidence-extraction operator Φ maps a trace set $\mathcal{D}$ to

$$\mathcal{E} = \Phi(\mathcal{D}) = \{\varepsilon_j = (\rho_j, \delta_j, u_j, \mathcal{I}_j)\}_{j=1}^J, \quad (5)$$

where ε_j denotes the j th item of evolution evidence and J denotes the number of evidence items; $\rho_j \in \{+, -\}$ denotes the direction of the evidence: positive evidence indicates that the relevant behavior pattern is consistent with improved performance and may support Loop Policy reuse, whereas negative evidence indicates that the relevant behavior pattern is consistent with failure or performance degradation and may support Loop Policy revision; δ_j denotes a metric change relative to a control run or a statistical reference for comparable traces; u_j denotes the Loop Policy component targeted by the evidence, taking a value corresponding to observation, planning, memory, action, verification, recovery,

stopping, or budget control as defined in the next subsection; and $\mathcal{I}_j \subseteq \{i \mid \tau_i \in \mathcal{D}\}$ denotes the set of global trace indices supporting the judgment.

In summary, the trace and evidence contract transforms historical interactions into attributable and verifiable evolution evidence that can be used to adjust the Loop Policy, laying the foundation for defining the modifiable and reusable Loop Policy object in the next subsection.

C. Loop Policy

Run evidence must act on an explicit loop-control object before it can be converted into reusable Loop Policy experience. Loop Engineering extends the scope of engineering to the external agent loop and systematically designs its objectives, feedback, verification, stopping, and cross-run experience [16]. Building on the preceding abstract type declaration, we define the set of external rules that controls how an agent advances through a complete execution loop as a Loop Policy, whose eight-component structure is denoted by

$$\pi = \langle \pi_{\text{obs}}, \pi_{\text{plan}}, \pi_{\text{mem}}, \pi_{\text{act}}, \pi_{\text{ver}}, \pi_{\text{rec}}, \pi_{\text{stop}}, \pi_{\text{bud}} \rangle \in \Pi, \quad (6)$$

where Π denotes the Loop Policy space; π_{obs} governs the acquisition and filtering of observations, π_{plan} governs plan generation and updating, π_{mem} governs memory writing, retrieval, and use, π_{act} governs action and tool selection, π_{ver} governs outcome verification, π_{rec} governs recovery from failure, π_{stop} governs the loop's stopping conditions, and π_{bud} governs resource-budget allocation and adjustment. Together, these eight components determine the state-update relation f_π and action-decision relation h_π in Eq. (2), allowing a Loop Policy to cover the complete decision process from incorporating environment feedback into the control state to producing the next action.

Fig. 2 presents a simplified Loop Policy example in a YAML-style format. This representation organizes the eight categories of loop rules into a readable and modifiable external structure, allowing Loop Policy content to be stored, compared, and reused independently of any single-run context.

To support the reuse of loop-control experience across tasks, we design a Loop Policy as an external asset independent of a specific host agent and use a unified representation to carry its control semantics. A Loop Policy can be integrated as a pluggable object into different agent frameworks through an adaptation interface, without binding its content to a particular host implementation, thereby supporting Loop Policy replacement, comparison, and evolution across agent frameworks. This externalized design provides the basis for the versioned asset representation in the next subsection.

D. Asset Representation of a Loop Policy

As a control object that can be independently compared and reused, a Loop Policy also requires a stable asset representation. Reusing a Loop Policy across runs requires the Loop Policy itself, its applicability conditions, and its version provenance to be jointly identifiable. We represent the asset

loop-policy-v2.1

observation: select task-relevant feedback
planning: update plan after each observation
memory: retrieve reusable trace evidence
action: choose tool or agent action
verification: require external evidence
recovery: retry | reroute | replan
stopping: verified | budget exhausted
budget control: allocate calls, time, and tokens

Fig. 2. A YAML-style example of a Loop Policy. The boldface fields correspond to the eight categories of loop-control rules: observation, planning, memory, action, verification, recovery, stopping, and budget control.

corresponding to the v th registered Bundle version as a Bundle:

$$\mathcal{B}_v = \langle \pi_v, \chi_v, \ell_v \rangle, \quad (7)$$

where v is the stable version index of a registered Bundle, π_v denotes the Loop Policy of this version, $\chi_v \subseteq \mathcal{Z}$ denotes the set of applicability conditions for this Loop Policy, and ℓ_v denotes the version provenance, including the stable index p_v of the direct parent version and the evolution evidence on which the version is based; the initial Bundle has $p_v = \perp$ and no supporting evolution evidence. Together, these three components describe which Loop Policy is used, under what conditions it is executed, and its version provenance, allowing the Loop Policy to be stored and reused as a complete object. Registration assigns a stable index when a valid candidate Bundle is formed so that it can enter evaluation history and lineage; registration itself does not mean that the candidate has passed the gate, been released, or been activated. Hereafter, k is reserved for evolution rounds or offline generation indices and no longer serves as a stable version identity.

Bundles are compared under shared applicability conditions $z \in \chi_{\text{par}} \cap \chi_{\text{cand}}$, where the subscripts par and cand denote the parent and candidate versions, respectively, and π_{par} and π_{cand} are the corresponding Loop Policies. For each noninitial version with $p_v \neq \perp$, the version provenance ℓ_v links it to its direct parent version and to the run evidence supporting the change. The directed relationships from parent versions to successor versions form a policy lineage, allowing historical experience to accumulate along version relationships and supporting subsequent candidate comparison, version selection, and outcome tracing.

This section has defined, in sequence, the loop interaction process, traces and evolution evidence, the Loop Policy, and its asset representation, beginning with the long-horizon complex task. These definitions specify the objects of self-evolution, the sources of experience, and the basis for comparison, providing a unified notation for the next section's presentation of online and offline evolution, the candidate lifecycle, and the robust release gate.

IV. OPENLOOPEVOLVE METHOD

Section III formally defined long-horizon complex tasks, traces, evolution evidence, and Loop Policies. Building on these definitions, OpenLoopEvolve converts traces accumulated across runs into a basis for modifying Loop Policies and optimizes an external Loop Policy rather than the parameters of the base model. Starting from the current Champion, the method constructs evidence, generates candidates, performs paired evaluation, and applies constrained release to produce a successor version. Depending on the evidence source and update timing, this common framework can be instantiated as either online or offline evolution.

A. Overall OpenLoopEvolve Framework

The two evolution modes of OpenLoopEvolve share the same processing chain from run experience to policy assets. Let $\mathcal{D}_k$ denote the set of traces used in evolution round k . The corresponding evidence-construction process is

$$\mathcal{E}_k = \Phi(\mathcal{D}_k), \quad \mathcal{D}_k \in \{\mathcal{D}_k^{\text{off}}, \mathcal{D}_k^{\text{on}}\}, \quad (8)$$

where Φ follows Eq. (5); $\mathcal{D}_k^{\text{off}}$ and $\mathcal{D}_k^{\text{on}}$ are attributable traces available from archives and online operation, respectively. The online mode retains the round index and uses $\mathcal{E}_k = \Phi(\mathcal{D}_k^{\text{on}})$; because the offline archive remains fixed across generations, $\Phi(\mathcal{D}_k^{\text{off}})$ is abbreviated as $\mathcal{E}$ below. A trace can enter $\mathcal{D}_k$ only if its outcome has been verified by $\mathcal{V}_{g_i}$ and its source belongs to $\mathcal{S}_{g_i}$. Candidate generation reads $\varepsilon_j = (\rho_j, \delta_j, u_j, \mathcal{I}_j)$, where u_j identifies a component in Eq. (6); $G_{\text{LLM}}^{\text{on}}$ returns one registered Bundle, whereas $G_{\text{LLM}}^{\text{off}}$ and $V_{\text{LLM}}^{\text{off}}$ return sets of registered Bundles; a proposal that fails to form a valid Bundle yields the empty value or is excluded from the set. A candidate may be released as a successor asset only after passing paired evaluation and the gate. The host activates a released policy only at a task boundary, and only the activated policy controls environment interaction according to Eq. (2) and produces new traces as defined in Eq. (4), thereby forming the closed loop shown in Fig. 1.

For readability, Table I summarizes the key symbols used throughout the problem formulation and method in the order of task and interaction, trace and evidence, policy assets, and evolution and release. Indices, temporary variables, and hyperparameters used only within a single equation or algorithm are defined locally.

Champion–Challenger paired evaluation provides a common comparison basis for both modes [18], [19]. Let the current Champion Bundle be $\mathcal{B}_c = \langle \pi_c, \chi_c, \ell_c \rangle$, and let candidate Bundle j generated in round k be $\mathcal{B}_{k,j} = \langle \pi_{k,j}, \chi_{k,j}, \ell_{k,j} \rangle \equiv \mathcal{B}_{v_{k,j}}$, where (k, j) is the candidate’s search coordinate and $v_{k,j}$ is the stable version index assigned when the candidate is formed. The two are executed separately only on the same task g_i and under the same controlled run condition $z_{i,j} \in \chi_c \cap \chi_{k,j}$. For pair i of candidate j , the value-score difference and relative change are defined as

$$\begin{aligned} \Delta_{i,j} &= q(\pi_{k,j}; g_i, z_{i,j}) - q(\pi_c; g_i, z_{i,j}), \\ r_{i,j} &= \frac{\Delta_{i,j}}{\max\{|q(\pi_c; g_i, z_{i,j})|, \epsilon\}}, \end{aligned} \quad (9)$$

where $\Delta_{i,j}$ is the candidate’s absolute value-score change relative to the Champion, $r_{i,j}$ is the relative change, and $\epsilon > 0$ ensures numerical stability when the Champion value score is close to zero. In each paired run, the Loop Policy in the corresponding Bundle controls the execution loop in Eq. (2). The value score q remains determined by $\mathcal{M}_{g_i}$ and $\mathcal{V}_{g_i}$ in the task contract, while the action scope and resource upper bounds comply with $\mathcal{U}_{g_i}$ and β_{g_i} , respectively. Holding the task, model, tools, and resource conditions constant controls for the primary external sources of variation, allowing paired changes to reflect the effect of Loop Policy modifications more directly. The valid relative changes for candidate j form the sample $r_j = \{r_{i,j}\}$ and, together with the verification risk $\mathbf{c}_{\mathcal{V}_{g_i}}$ in Eq. (3), resource consumption $\mathbf{b}$, failure states, and evidence validity, constitute the paired evaluation $\mathcal{P}_{k,j}$. For runs with termination failures or invalid evidence, the evaluation protocol uses a bounded failure-aware change and retains the corresponding validity and reliability states. Thus, $\text{PairEval}(\mathcal{B}_c, \mathcal{B}_{k,j})$ in the algorithms below always denotes paired evaluation of complete Bundles under shared applicability conditions, rather than a comparison of two bare policies without regard to applicability conditions.

Paired evaluation does not directly replace the current policy according to a single mean score; it provides evidence for constrained release. Let Γ_b , Γ_e , Γ_t , and Γ_c indicate whether the four constraint categories of benefit, evidence quality, tail risk, and resource cost are satisfied, respectively. The release gate can be expressed as

$$\begin{aligned} \text{Gate}(\mathcal{P}_{k,j}, \Theta) &= \text{accept} \\ \iff \prod_{\alpha \in \mathcal{J}_{\text{gate}}} \Gamma_{\alpha}(\mathcal{P}_{k,j}, \Theta_{\alpha}) &= 1, \end{aligned} \quad (10)$$

where $\mathcal{J}_{\text{gate}} = \{b, e, t, c\}$ corresponds to benefit, evidence quality, tail risk, and resource cost; $\Gamma_{\alpha}(\mathcal{P}_{k,j}, \Theta_{\alpha}) \in \{0, 1\}$ indicates whether the constraint of category α is satisfied; and $\Theta = \{\Theta_{\alpha}\}$ is the gate configuration. Γ_e checks the evidence permitted by $\mathcal{S}_{g_i}$, Γ_t enforces $\mathbf{c}_{\mathcal{V}_{g_i}} \preceq \kappa_{g_i}$, and Γ_c enforces $\mathbf{b} \preceq \beta_{g_i}$. The number of pairs, win rate, confidence interval, task failures, and tail losses may be used as internal evaluation quantities. A Challenger is eligible for release only if all enabled constraints are satisfied, and the configured thresholds cannot relax the verification and resource boundaries of the task contract.

Within this common framework, the online and offline modes are suited to different forms of available run experience. When the task distribution continues to change or new feedback must be incorporated promptly, the online mode can perform staged updates. When archived traces cover the main run states of the target task, the offline mode can conduct a concentrated search over multiple policy directions. The application selects the mode according to the scope of available traces, feedback timeliness, and update requirements; the system does not switch modes automatically. The modes differ in candidate-search procedures and update timing, while their policy comparison and release eligibility are governed uniformly by Eqs. (9) and (10).

The evaluation histories of the two modes are explicitly typed. Let $\mathcal{R} = \mathcal{R}^{\text{on}} \uplus \mathcal{R}^{\text{off}}$ be a disjoint union. Each record

TABLE I
MAIN NOTATION AND MEANINGS

Category	Symbol	Meaning
Task and interaction	$g = \langle d, \mathcal{U}_g, \mathcal{M}_g, \mathcal{S}_g, \mathcal{V}_g, \beta_g \rangle$	Executable contract for a long-horizon task, comprising its objective, permitted executors, evaluation metrics, evidence sources, verification protocol, and resource budget
	$x_t, o_t, a_t, e_t; f_\pi, h_\pi$	Control state, observation, action, and environment event at step t ; state-update and action-decision functions constrained by the policy
	$z \in \mathcal{Z}; q(\pi; g, z); \mathbf{c}_{\mathcal{V}_g}, \kappa_g; \mathbf{b}, \beta_g$	Controlled run condition; value score; verification risk and its upper bound; resource consumption and its upper bound
Traces and evidence	$\tau_i = (g_i, \pi_{v(i)}, \mathcal{H}_i, y_i)$	Attributable trace of run i , including the task, stable Bundle version, interaction history, and run outcome
	$\mathcal{D}, \mathcal{E} = \Phi(\mathcal{D})$	Set of attributable traces with verified outcomes and compliant evidence sources; the evolution-evidence set; and the evidence-extraction operator
	$\varepsilon_j = (\rho_j, \delta_j, u_j, \mathcal{I}_j)$	Direction, metric change, target policy component, and supporting trace indices of evolution-evidence item j
Policy assets	$\pi \in \Pi$	Loop Policy and its policy space; the policy comprises rules for observation, planning, memory, action, verification, recovery, stopping, and budget control
	$\mathcal{B}_v = \langle \pi_v, \chi_v, \ell_v \rangle$	Bundle corresponding to stable version index v , comprising a Loop Policy, a set of applicability conditions, and version provenance
	χ_v, ℓ_v	Bundle applicability conditions and version provenance containing the parent version and supporting evidence
Evolution and release	$\mathcal{D}^{\text{off}}, \mathcal{D}_k^{\text{on}}, \mathcal{E}, \mathcal{E}_k$	Archived offline traces, traces visible in online round k , and their corresponding evolution evidence
	$G_{\text{LLM}}^{\text{on}}, G_{\text{LLM}}^{\text{off}}, V_{\text{LLM}}^{\text{off}}$	Online single-candidate generation, offline initial-candidate-set generation, and offline candidate mutation and recombination operators
	$\mathcal{R} = \mathcal{R}^{\text{on}} \uplus \mathcal{R}^{\text{off}}$	Mode-typed evaluation histories; the online component records candidates, evaluations, and release decisions, whereas the offline component records candidate sets and evaluation mappings
	$\mathcal{B}_c, \mathcal{B}_{k,j} \equiv \mathcal{B}_{v_{k,j}}$	Champion and Challenger Bundle j in round k ; $v_{k,j}$ is its stable version index (online: $j = 1, \mathcal{B}'_k \equiv \mathcal{B}_{k,1}$)
	$\Delta_{i,j}, r_{i,j}, \mathcal{P}_{k,j}$	Absolute and relative value-score changes for pair i , and the complete paired evaluation of a candidate
	$\Gamma_\alpha, \Theta_\alpha, \text{Gate}$	Release constraint of category α , its configuration, and the aggregate release-gate operator
	$\mathcal{C}_k, \mathcal{F}_k, \overline{\mathcal{P}}_k$	Candidate set, retained set comprising the fixed Champion and selected nondominated candidates, and cumulative paired-evaluation mapping at offline generation k

in the online component $\mathcal{R}^{\text{on}}$ is $(\mathcal{B}', \mathcal{P}, \omega)$, which stores a candidate Bundle, its paired evaluation, and the release decision, respectively. The record for offline generation k in $\mathcal{R}^{\text{off}}$ is $(\mathcal{C}_k, \mathcal{P}_k)$, which stores the candidate set and its evaluation mapping. Each generator reads only the history component of its own mode, preventing the two record structures from being interpreted as the same untyped tuple.

B. Online Loop Policy Evolution

Online evolution uses a continuous stream of recent feedback to update the Loop Policy in stages, allowing subsequent tasks to incorporate the latest run experience. At online update time k , the host submits completed traces attributable to explicit policy versions. Once the application-defined update condition is satisfied, these traces are converted into evolution evidence $\mathcal{E}_k$. The large language model then generates a Challenger Loop Policy and encapsulates it in a candidate Bundle:

$$\mathcal{B}'_k = G_{\text{LLM}}^{\text{on}}(\mathcal{B}_k, \mathcal{E}_k, \mathcal{R}_{<k}^{\text{on}}) = \langle \pi'_k, \chi'_k, \ell'_k \rangle, \quad (11)$$

where $\mathcal{B}_k = \langle \pi_k, \chi_k, \ell_k \rangle$ is the current Champion held at online update time k , and $\mathcal{R}_{<k}^{\text{on}}$ is the prior online evaluation history; here, k denotes an update time, not a stable version index. The online mode has one candidate per round, for which we set $j = 1$. $G_{\text{LLM}}^{\text{on}}$ rewrites π'_k , sets $\chi'_k \subseteq \chi_k$, and records the parent version and evidence from the current round in ℓ'_k . When a valid candidate is formed, stable version index $v_{k,1}$ is assigned atomically and the candidate is written as $\mathcal{B}'_k \equiv \mathcal{B}_{k,1} \equiv \mathcal{B}_{v_{k,1}}$, thereby directly producing a registered candidate Bundle as defined in Eq. (7). The candidate undergoes paired evaluation only within $\chi_k \cap \chi'_k$. If it passes the gate, it is recorded as a released successor asset and activated at a subsequent task boundary as $\mathcal{B}_{k+1} = \mathcal{B}'_k$; otherwise, $\mathcal{B}_{k+1} = \mathcal{B}_k$. To emphasize the processing sequence of a single update, Algorithm 1 omits the update subscript k within the loop body; $\mathcal{B}$, $\mathcal{D}^{\text{on}}$, $\mathcal{E}$, and $\mathcal{B}'$ correspond respectively to $\mathcal{B}_k$, $\mathcal{D}_k^{\text{on}}$, $\mathcal{E}_k$, and $\mathcal{B}'_k$ for the current batch.

Online updates must preserve policy consistency within each task and support rollback after release. A task is con-

trolled by the same activated Champion from start to finish. A Challenger that passes the gate may be released but is activated only at a subsequent task boundary, so that run outcomes remain attributable to explicit policy versions. After the new version is activated, CANARYMONITOR monitors its performance using attributable subsequent feedback. When the degradation condition is met, it rolls back to the parent version and quarantines the traces that triggered the rollback from the current update batch; other usable traces in the same batch may still participate in candidate generation. The update condition Trig may depend on the amount of feedback, an update period, or a task stage. Its only function is to determine when evolution begins; it does not replace the release gate in the overall framework. The online mode therefore forms a closed loop of feedback triggering, candidate validation, release, subsequent-task activation, and degradation rollback, as detailed in Algorithm 1.

Algorithm 1 Online Loop Policy Optimization

Input: Champion Bundle $\mathcal{B} = \langle \pi_c, \chi, \ell \rangle$, stream of attributable traces $\{\tau_i\}$ (with y_i verified by $\mathcal{V}_{g_i}$ and its evidence source belonging to $\mathcal{S}_{g_i}$), update condition Trig, online evaluation history $\mathcal{R}^{\text{on}}$, gate configuration Θ
Output: Champion $\mathcal{B}$ updated with the trace stream and online evaluation history $\mathcal{R}^{\text{on}}$

- ▷ Continuously receive batches of traces submitted by the host when checkpoint or feedback conditions are met.
- 1: **while** new trace batches continue to arrive **do**
- 2: $\mathcal{D}^{\text{on}} \leftarrow \text{RECEIVE_TRACES}$
 - ▷ Monitor the performance of the new version; roll back a degraded version, and let $\mathcal{D}^{\text{on}}$ denote the usable traces after quarantining the corresponding feedback.
- 3: $(\mathcal{B} = \langle \pi_c, \chi, \ell \rangle, \mathcal{D}^{\text{on}}) \leftarrow \text{CANARYMONITOR}(\mathcal{B}, \mathcal{D}^{\text{on}})$
 - ▷ Start evolution only when usable feedback exists and the external update condition is satisfied.
- 4: **if** $\mathcal{D}^{\text{on}} = \emptyset$ **or** $\neg \text{Trig}(\mathcal{D}^{\text{on}})$ **then continue**
 - ▷ Derive the evolution evidence for the current round from recent traces.
- 5: $\mathcal{E} \leftarrow \Phi(\mathcal{D}^{\text{on}})$
 - ▷ Use the large language model to form a valid registered Challenger Bundle; registration atomically assigns its stable index (online $j = 1$) before evaluation or history insertion.
- 6: $\mathcal{B}' = \langle \pi', \chi', \ell' \rangle \leftarrow G_{\text{LLM}}^{\text{on}}(\mathcal{B}, \mathcal{E}, \mathcal{R}^{\text{on}})$
 - ▷ Wait for the next feedback batch if no testable candidate is formed.
- 7: **if** $\mathcal{B}' = \emptyset$ **then continue**
 - ▷ Perform paired evaluation of the Bundles under shared conditions in $\chi \cap \chi'$, and let ω denote the release-gate decision.
- 8: $\mathcal{P} \leftarrow \text{PAIREVAL}(\mathcal{B}, \mathcal{B}')$, $\omega \leftarrow \text{GATE}(\mathcal{P}, \Theta)$
 - ▷ Record the candidate, evaluation result, and release decision for the current round.
- 9: $\mathcal{R}^{\text{on}} \leftarrow \mathcal{R}^{\text{on}} \cup \{(\mathcal{B}', \mathcal{P}, \omega)\}$
 - ▷ Mark a gate-passing candidate as the released successor asset and activate it as the current Champion only at a subsequent task boundary.
- 10: **if** $\omega = \text{accept}$ **then** $\mathcal{B} \leftarrow \mathcal{B}'$
- 11: **end while**
 - ▷ Return the policy asset and evaluation history retained after online updates.
- 12: **return** $\mathcal{B}, \mathcal{R}^{\text{on}}$

C. Offline Loop Policy Evolution

Offline evolution takes archived traces and the current Champion as input, performs a multi-candidate, multi-generation evaluation–selection–generation search outside the task-execution path, and releases a candidate Bundle that passes the release gate as the successor Champion.

Archived traces $\mathcal{D}^{\text{off}}$ yield evolution evidence $\mathcal{E}$ through Eq. (8). Based on this evidence, the large language model autonomously proposes candidates around the Champion and updates the search direction according to evaluations from previous generations. To preserve the policy identity, applicability conditions, and version provenance defined in Eq. (7)

throughout the search, candidates are not stored as bare policies. Let K be the total number of evolution generations, $\mathcal{C}_k$ the set of candidate Bundles at generation k , $\mathcal{F}_k$ the retained set comprising the fixed Champion and the selected nondominated candidates through generation k , and $\overline{\mathcal{P}}_k$ the cumulative paired-evaluation mapping through that generation. The offline search can then be written as

$$\begin{aligned}
 \mathcal{F}_0 &= \{\mathcal{B}\}, \\
 \overline{\mathcal{P}}_0 &= \emptyset, \\
 \mathcal{C}_1 &= G_{\text{LLM}}^{\text{off}}(\mathcal{B}, \mathcal{E}, \mathcal{R}_0^{\text{off}}), \\
 \mathcal{P}_k &= \{\mathcal{B}_{k,j} \mapsto \mathcal{P}_{k,j}\}_{\mathcal{B}_{k,j} \in \mathcal{C}_k}, \\
 \mathcal{P}_{k,j} &= \text{PairEval}(\mathcal{B}, \mathcal{B}_{k,j}), \\
 \overline{\mathcal{P}}_k &= \overline{\mathcal{P}}_{k-1} \cup \mathcal{P}_k, \\
 \mathcal{A}_k &= (\mathcal{F}_{k-1} \setminus \{\mathcal{B}\}) \cup \mathcal{C}_k, \\
 \mathcal{F}_k &= \{\mathcal{B}\} \cup \text{Select}(\mathcal{A}_k, \overline{\mathcal{P}}_k), \\
 \mathcal{C}_{k+1} &= V_{\text{LLM}}^{\text{off}}(\mathcal{F}_k, \mathcal{E}, \mathcal{R}_k^{\text{off}}).
 \end{aligned} \tag{12}$$

where $\mathcal{B} = \langle \pi_c, \chi_c, \ell_c \rangle$ is the fixed Champion; $\mathcal{P}_k$ is the evaluation mapping for the current generation, keyed by candidate Bundles with each candidate evaluation $\mathcal{P}_{k,j}$ as its value; $\overline{\mathcal{P}}_k$ is the cumulative evaluation mapping obtained by merging each generation into the initially empty mapping $\overline{\mathcal{P}}_0$; and $\mathcal{R}_k^{\text{off}}$ is the offline evaluation history. Each formed candidate is registered with a stable version index according to $\mathcal{B}_{k,j} \equiv \mathcal{B}_{v_{k,j}}$, so merging the mappings does not conflate Bundles from different generations. $\mathcal{A}_k$ collects retained candidates from previous generations and new candidates from the current generation but excludes the fixed Champion. Consequently, Select queries only candidates in the domain of the evaluation mapping, after which their union with $\mathcal{B}$ forms $\mathcal{F}_k$. $V_{\text{LLM}}^{\text{off}}$ generates the next generation when $k < K$. A candidate j produced through recombination at generation k designates one direct parent version in $\ell_{k,j}$, while other donors are recorded in the evolution evidence; its applicability conditions are restricted to those of the direct parent version and the evidence coverage. This relation preserves effective search directions and their provenance across generations.

Paired evaluation provides feedback for generational updates. Each generation’s candidate Bundles are compared with the Champion on shared tasks and run conditions within the intersection of their applicability conditions and χ_c . The definitions of value scores, verification risk, and resource consumption continue to follow the task contract in Section III and Eq. (3). Benefit, reliability, tail performance, and resource cost jointly determine retention. The large language model accordingly modifies control rules or recombines complementary rules from different policies, while failures exposed by eliminated candidates also enter the evaluation history. The Champion remains a stable reference, allowing successes and failures to jointly constrain the search direction of the next generation.

Nondominated retention across generations determines only the subsequent search; formal release remains subject to Eqs. (3) and (10). After K generations of evolution, let $\mathcal{F}_K^{\text{acc}}$ denote the candidate Bundles in the final retained set that are eligible for release. ValueSelect selects the candidate

with the highest value score aggregated by the evaluation protocol, using reliability, tail performance, and resource cost for nondominated tie-breaking only when value scores are tied, and makes it the successor Champion $\mathcal{B}^+ = \mathcal{B}^*$. If $\mathcal{F}_K^{\text{acc}}$ is empty, the original Champion remains unchanged. Algorithm 2 gives the procedure.

Algorithm 2 Offline Loop Policy Optimization

Input: Champion Bundle $\mathcal{B} = \langle \pi_c, \chi_c, \ell_c \rangle$, archived traces $\mathcal{D}^{\text{off}} = \{\tau_i\}$ (with y_i verified by $\mathcal{V}_{g_i}$ and its evidence source belonging to $\mathcal{S}_{g_i}$), initial offline evaluation history $\mathcal{R}_0^{\text{off}}$, number of evolution generations K , gate configuration Θ

Output: Successor Champion $\mathcal{B}^+$ and offline evaluation history $\mathcal{R}_K^{\text{off}}$ after evolution

- 1: $\mathcal{E} \leftarrow \Phi(\mathcal{D}^{\text{off}})$
 - ▷ Derive evidence related to Loop Policy modifications from archived runs.
 - ▷ Use the large language model to form the initial valid candidate Bundle set; each candidate is atomically assigned stable index $v_{1,j}$ before entering the evaluation mapping.
- 2: $\mathcal{C}_1 \leftarrow G_{\text{LLM}}^{\text{off}}(\mathcal{B}, \mathcal{E}, \mathcal{R}_0^{\text{off}})$
 - ▷ Initialize the elite set and cumulative evaluations with the complete Champion Bundle.
- 3: $\mathcal{F}_0 \leftarrow \{\mathcal{B}\}, \bar{\mathcal{P}}_0 \leftarrow \emptyset$
 - ▷ Advance the multi-generation search through paired evaluation and evidence-guided policy generation.
- 4: **for** $k = 1, \dots, K$ **do**
 - ▷ Perform Bundle paired evaluation for each candidate under applicability conditions shared with χ_c , and construct the evaluation mapping for the current generation.
- 5: $\mathcal{P}_k \leftarrow \emptyset$
- 6: **for all** $\mathcal{B}_{k,j} \in \mathcal{C}_k$ **do**
- 7: $\mathcal{P}_k(\mathcal{B}_{k,j}) \leftarrow \text{PAIREVAL}(\mathcal{B}, \mathcal{B}_{k,j})$
- 8: **end for**
 - ▷ Accumulate the evaluations for the current generation and retain competitive policy directions from existing elite Bundles and new candidates.
- 9: $\mathcal{R}_k^{\text{off}} \leftarrow \mathcal{R}_{k-1}^{\text{off}} \cup \{(\mathcal{C}_k, \mathcal{P}_k)\}, \bar{\mathcal{P}}_k \leftarrow \bar{\mathcal{P}}_{k-1} \cup \mathcal{P}_k$
- 10: $\mathcal{A}_k \leftarrow (\mathcal{F}_{k-1} \setminus \{\mathcal{B}\}) \cup \mathcal{C}_k$
- 11: $\mathcal{F}_k \leftarrow \{\mathcal{B}\} \cup \text{SELECT}(\mathcal{A}_k, \bar{\mathcal{P}}_k)$
- 12: **if** $k < K$ **then**
 - ▷ Use the large language model to mutate and recombine policies from retained Bundles and cumulative feedback; each newly formed valid Bundle is assigned stable index $v_{k+1,j}$ before next-generation evaluation.
- 13: $\mathcal{C}_{k+1} \leftarrow V_{\text{LLM}}^{\text{off}}(\mathcal{F}_k, \mathcal{E}, \mathcal{R}_k^{\text{off}})$
- 14: **end if**
- 15: **end for**
 - ▷ Collect candidates from the final retained Bundles that pass the robust release gate.
- 16: $\mathcal{F}_K^{\text{acc}} \leftarrow \{\mathcal{B}' \in \mathcal{F}_K \setminus \{\mathcal{B}\} \mid \text{GATE}(\bar{\mathcal{P}}_K(\mathcal{B}'), \Theta) = \text{accept}\}$
 - ▷ Retain the original Champion if no candidate is eligible for release.
- 17: **if** $\mathcal{F}_K^{\text{acc}} = \emptyset$ **then** $\mathcal{B}^+ \leftarrow \mathcal{B}$ **and return** $\mathcal{B}^+, \mathcal{R}_K^{\text{off}}$
 - ▷ Select the qualified Bundle with the highest aggregate value and release it in full as the successor version of the Champion.
- 18: $\mathcal{B}^* \leftarrow \text{VALUESELECT}(\mathcal{F}_K^{\text{acc}}, \bar{\mathcal{P}}_K), \mathcal{B}^+ \leftarrow \mathcal{B}^*$
- 19: **return** $\mathcal{B}^+, \mathcal{R}_K^{\text{off}}$

V. EXPERIMENTAL RESULTS

A. YC-Bench Results

We evaluate the effect of OpenLoopEvolve on long-horizon task execution using YC-Bench [6]. The benchmark requires an agent to operate a business continuously over a simulated year and uses final funds to measure the cumulative effects of sequential decisions, including task selection, resource allocation, and risk management. We designate as the Baseline the Native agent that uses the official YC-Bench Agent Loop but neither integrates OpenLoopEvolve nor loads an additional Loop Policy, thereby measuring model capability under the benchmark’s native execution framework. To separate the effect of introducing a policy from that of evolving it, Fixed- π_0 loads the same initial Loop Policy as the two OLE modes

but keeps it unchanged throughout the year. OLE-online starts from π_0 and evolves and updates the policy online during execution, making 12 candidate-update attempts in total; OLE-offline uses a fixed policy obtained through offline evolution before the experiment and does not update it during evaluation. At each trigger, offline evolution uses the large language model to autonomously generate $k=3$ candidates, with a pre-registered maximum of 4 candidate rounds. All four settings use deepseek-v4-flash [20] under the official medium configuration, seeds 1/2/3, and a 20-turn context, and begin the annual evaluation from the same initial state.

Table II summarizes the annual results for the four settings. Relative to Fixed- π_0 , OLE-online and OLE-offline increase mean final funds by 140.07% and 166.31%, respectively, and increase task success rate by 13.98 and 17.92 percentage points. Annual survival improves from 1/3 to 2/3 and 3/3, while mean maximum drawdown decreases by 10.77% and 38.30%, respectively. This comparison shows that both online and offline evolution can produce a more effective Loop Policy from the initial Loop Policy, with consistent advantages in long-term returns, task success rate, annual survival, and risk metrics. Fig. 3 shows the fund trajectories of the four settings over the simulated year.

B. Token Usage and Evolution Cost

To measure changes in inference resources after introducing OLE, we aggregate the input and output tokens recorded for each model call and use their sum as token usage. All four settings include the official seeds 1/2/3. Baseline and Fixed- π_0 generate only main-task execution calls, whereas OLE-online and OLE-offline also incur candidate-Champion evolution-validation calls through paired evaluation. Table III(a) reports main-task tokens, mean survival duration, and mean token usage per call, and computes the changes in the average relative to Baseline and Fixed- π_0 . Table III(b) compares the main-task total, evolution-validation cost, and observable total across the four settings.

Main-task execution for Baseline, Fixed- π_0 , OLE-online, and OLE-offline comprises 1108, 799, 1575, and 1907 model calls, respectively, with mean survival durations of 243.00, 207.78, 294.00, and 365.00 days. The corresponding mean usage per call is 25001, 23496, 22029, and 18676 tokens. Relative to Baseline, mean usage decreases by 11.89% and 25.30% for OLE-online and OLE-offline, respectively; relative to Fixed- π_0 , it decreases by 6.24% and 20.51%, respectively. Although the two OLE modes have higher main-task token totals, they also support longer execution horizons and more task calls while using fewer tokens per call on average. This indicates that, in this experiment, the Loop Policy expands the scale of sustained task execution while keeping the inference cost per interaction under control.

The online mode attempts 12 candidates, with an evolution-validation cost of 29.82M tokens and an observable total of 64.52M tokens after adding main-task usage. The offline mode has an evolution-validation cost of 24.02M tokens and an observable total of 59.63M tokens. Baseline and Fixed- π_0 perform no evolution, so their observable totals equal their

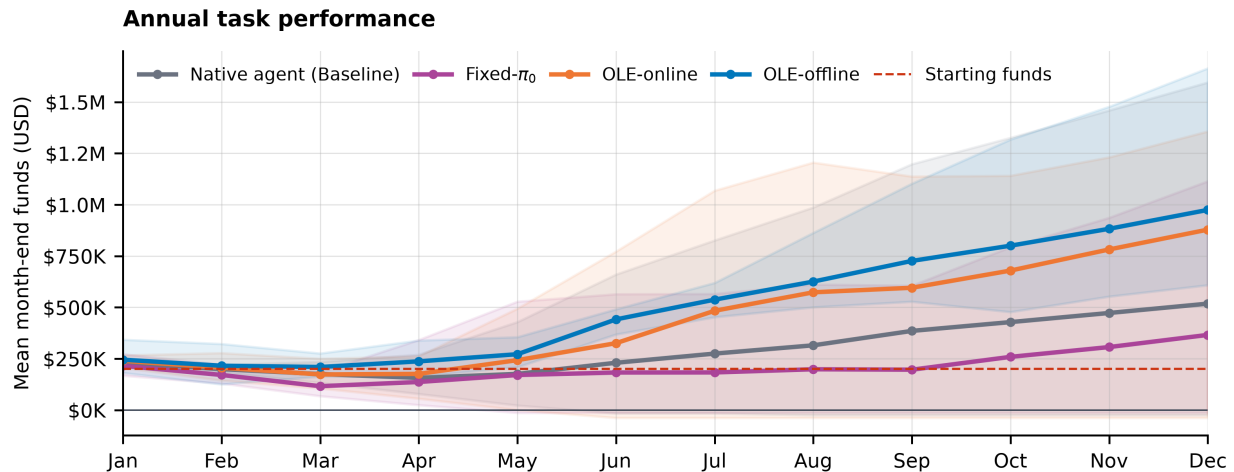


Fig. 3. Annual task performance of the four settings on YC-Bench. Curves and shaded regions show mean month-end funds and cross-seed ranges over three seeds, respectively; after early termination, terminal funds are carried forward only to retain a fixed aggregation denominator, not to indicate continued execution.

TABLE II
OVERALL RESULTS ON YC-BENCH.

Method	Mean final funds (\$)	Δ_{Native}	Δ_{π_0}	Task success rate	Annual survival	Mean maximum drawdown (\$)
Native agent (Baseline)	518,158.46	—	—	77.23%	1/3	191,462.39
Fixed initial policy (Fixed- π_0)	365,976.30	−29.37%	—	73.89%	1/3	187,022.86
OLE-online	878,601.79	+69.56%	+140.07%	87.87%	2/3	166,878.69
OLE-offline (fixed policy)	974,627.52	+88.09%	+166.31%	91.80%	3/3	115,402.09

Δ_{Native} and Δ_{π_0} use the Native agent and the fixed initial policy as their respective references.

TABLE III
MAIN-TASK TOKEN USAGE AND EVOLUTION-VALIDATION COST OF OLE.

(a) Main-task execution					
Method	Total (M tokens)	Mean survival duration (days)	Average per call (tokens)	Δ_{Base} (%)	Δ_{Fixed} (%)
Baseline	27.70	243.00	25,001	—	—
Fixed- π_0	18.77	207.78	23,496	−6.02%	—
OLE-online	34.70	294.00	22,029	−11.89%	−6.24%
OLE-offline	35.61	365.00	18,676	−25.30%	−20.51%

(b) Main-task and evolution costs			
Method	Main task (M tokens)	Evolution (M tokens)	Total (M tokens)
Baseline	27.70	0.00	27.70
Fixed- π_0	18.77	0.00	18.77
OLE-online	34.70	29.82	64.52
OLE-offline	35.61	24.02	59.63

(a) “Total” is the sum of main-task tokens across the three official seeds, in millions; “Mean survival duration” is reported in days; and “Average per call” is the main-task token usage divided by the number of model calls. Δ_{Base} and Δ_{Fixed} denote changes in this average relative to Baseline and Fixed- π_0 , respectively.

(b) Main-task totals, candidate–Champion evolution-validation costs, and their sums for the four settings, all in millions of tokens. Baseline and Fixed- π_0 perform no evolution and therefore incur zero evolution cost; the OLE totals exclude candidate-generation calls whose usage was not retained.

main-task usage of 27.70M and 18.77M tokens, respectively. In view of the improvements in sustained execution and overall task performance, the additional evolution cost is acceptable at the scale of the current experiment. Future work will further examine whether Loop Policies remain effective across

different tasks and operating environments.

VI. CONCLUSION

This paper presents OpenLoopEvolve, which represents an agent’s execution loop as an externalized Loop Policy that is versioned, traceable, and reusable. The framework constructs evolution evidence from run traces, uses a large language model to generate candidates, and governs policy updates through Champion–Challenger paired evaluation and robust gating. Its online and offline modes draw on recent feedback and archived experience, respectively, allowing loop improvements to accumulate beyond a single context.

Experiments on YC-Bench show that both evolution modes improve long-horizon task performance over a fixed initial policy while supporting longer effective execution with controlled inference cost per interaction. These results point to a broader role for Loop Policies as independent assets: they can be refined continually from run evidence and may be transferred and reused across tasks, agent hosts, and operating environments. Future work will evaluate this cross-scenario validity, evolution stability, and cost control at a larger scale.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>

- [2] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://arxiv.org/abs/2308.03688>
- [3] C. Ma, J. Zhang, Z. Zhu, C. Yang, Y. Yang, Y. Jin, Z. Lan, L. Kong, and J. He, "AgentBoard: An analytical evaluation board of multi-turn LLM agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: <https://arxiv.org/abs/2401.13178>
- [4] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A realistic web environment for building autonomous agents," in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://arxiv.org/abs/2307.13854>
- [5] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, " τ -bench: A benchmark for tool-agent-user interaction in real-world domains," in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://arxiv.org/abs/2406.12045>
- [6] M. He, A. Jain, A. Kumar, V. Tu, S. Bakshi, S. Patro, and N. Rajani, "YC-Bench: Benchmarking AI agents for long-term planning and consistent execution," *arXiv preprint arXiv:2604.01212*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.01212>
- [7] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. [Online]. Available: <https://doi.org/10.1145/3586183.3606763>
- [8] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html
- [9] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM agents are experiential learners," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 632–19 642. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/29936>
- [10] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research*, 2024. [Online]. Available: <https://openreview.net/forum?id=ehfRiFOR3a>
- [11] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. V. A., S. Haq, A. Sharma, T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into self-improving pipelines," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/f1cf02ce09757f57c3b93c0db83181e0-Abstract-Conference.html
- [12] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://arxiv.org/abs/2408.08435>
- [13] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, "AFlow: Automating agentic workflow generation," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/5492ecbce4439401798dcd2c90be94cd-Abstract-Conference.html
- [14] Y. Shang, Y. Li, K. Zhao, L. Ma, J. Liu, F. Xu, and Y. Li, "AgentSquare: Automatic LLM agent search in modular design space," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/0ae94013da7cd459402fd77874e09ee3-Abstract-Conference.html
- [15] Y. Guo, J. Lin, H. Wang, Y. Han, S. Hu, Z. Ni, L. Wang, and M. Chen, "SE-Agent: Self-evolution trajectory optimization in multi-step reasoning with LLM-based agents," in *Advances in Neural Information Processing Systems*, vol. 38, 2025. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/a911e543a95493ae5004fdc01909043e-Abstract-Conference.html
- [16] S. Macedo, "Stop hand-holding your coding agent: Engineering the loops that replace step-by-step prompting," *arXiv preprint arXiv:2607.00038*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.00038>
- [17] J. Huang, J. Hsia, J. Sun, F. Shi, W. Huang, and I. H. White, "Proof-or-stop: Don't trust the agent, trust the evidence—loop engineering for verifiable evidence-gated lifecycle control," *arXiv preprint arXiv:2607.14890*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.14890>
- [18] S. V. Nath, "Champion-challenger based predictive model selection," in *Proceedings 2007 IEEE SoutheastCon*, 2007, pp. 254–254. [Online]. Available: <https://doi.org/10.1109/SECON.2007.342897>
- [19] Q. Wu, C. Wang, J. Langford, P. Mineiro, and M. Rossi, "ChaCha for online AutoML," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 11 263–11 273. [Online]. Available: <https://proceedings.mlr.press/v139/wu21d.html>
- [20] DeepSeek-AI, "DeepSeek-V4: Towards highly efficient million-token context intelligence," *arXiv preprint arXiv:2606.19348*, 2026. [Online]. Available: <https://arxiv.org/abs/2606.19348>